REAL – TIME CHAT APP

Abstract

This paper presents the design and implementation of a real-time chat platform for instant private messaging. The proposed system enables users to register, authenticate securely, manage user profiles, and engage in one-to-one communication with minimal latency

Kevin Hirosh

Kevinhirosh8@gmail.com

BSIT241021

Table of Contents

No.	Section Title	Page No.
1	Introduction	3
2	Project Overview	3
3	Functional Requirements	3
4	Non-Functional Requirements	3
5	Main Module	4
6	Module Inputs & Outputs	5
7	Use Cases	5
8	Technical Specifications	6
9	Use Case Diagram	6
10	Project Timeline (Gantt Chart)	7
11	Technical Solution	7
12	Conclusion	7

This project delivers a secure, fast, and user-friendly real-time chat website where users can register, log in, manage profiles, see available users, and exchange private messages instantly.
The platform uses a React frontend, PHP REST APIs, MySQL database, and a WebSocket server for real-time communication.

Project Objectives

1. To Build a responsive web chat application for desktop and mobile use.
2. Enable real-time private messaging with minimal delay.
3. Provide secure authentication and profile management.
4. Store and retrieve chat history reliably.
5. Create a maintainable codebase for future features (group chat, notifications, file sharing).

Functional Requirements

1. Users can create accounts and authenticate securely.
2. Authenticated users can see other users and open chats.
3. Users can send/receive private messages in real time.
4. Messages are stored and retrievable in chronological order.
5. User status updates reflect connection state.
6. Users can update profile details and profile image.

Non-Functional Requirements

1. Real-time latency target: under 500ms in local/regional deployment.
2. Availability target: 99.5% for production deployment.
3. Security: OWASP-aligned validation and prepared statements.
4. Scalability: support at least 500 concurrent socket connections in initial release target.

Main Module

1. Authentication Module
    - User registration
    - User login/logout
    - Session/token handling
    - Password validation and security checks
2. User Management Module
    - User list retrieval
    - User online/offline status
    - Last seen tracking
    - Basic user profile data
3. Profile Module
    - View and update profile
    - Profile image upload and display
    - Profile validation (image type/size, required fields)
4. Chat/Messaging Module
    - One-to-one conversation selection
    - Send message
    - Receive message
    - Load of conversation history
    - Message states (sent/read)

Module Inputs & Outputs

Module	Inputs	Outputs
Authentication	email, password, username (register), HTTP request	success/fail response, user id, auth token/session, error messages
User Management	authenticated user id, query params (search/filter), status updates	user list, user status (online/offline), last seen, user summary data
Profile	ser id, profile form data, profile image file	updated profile record, image URL/path, validation errors, success message
Chat/Messaging	sender id, receiver id, message text/type, conversation request params	stored message id, conversation history array, delivery/read state, error response

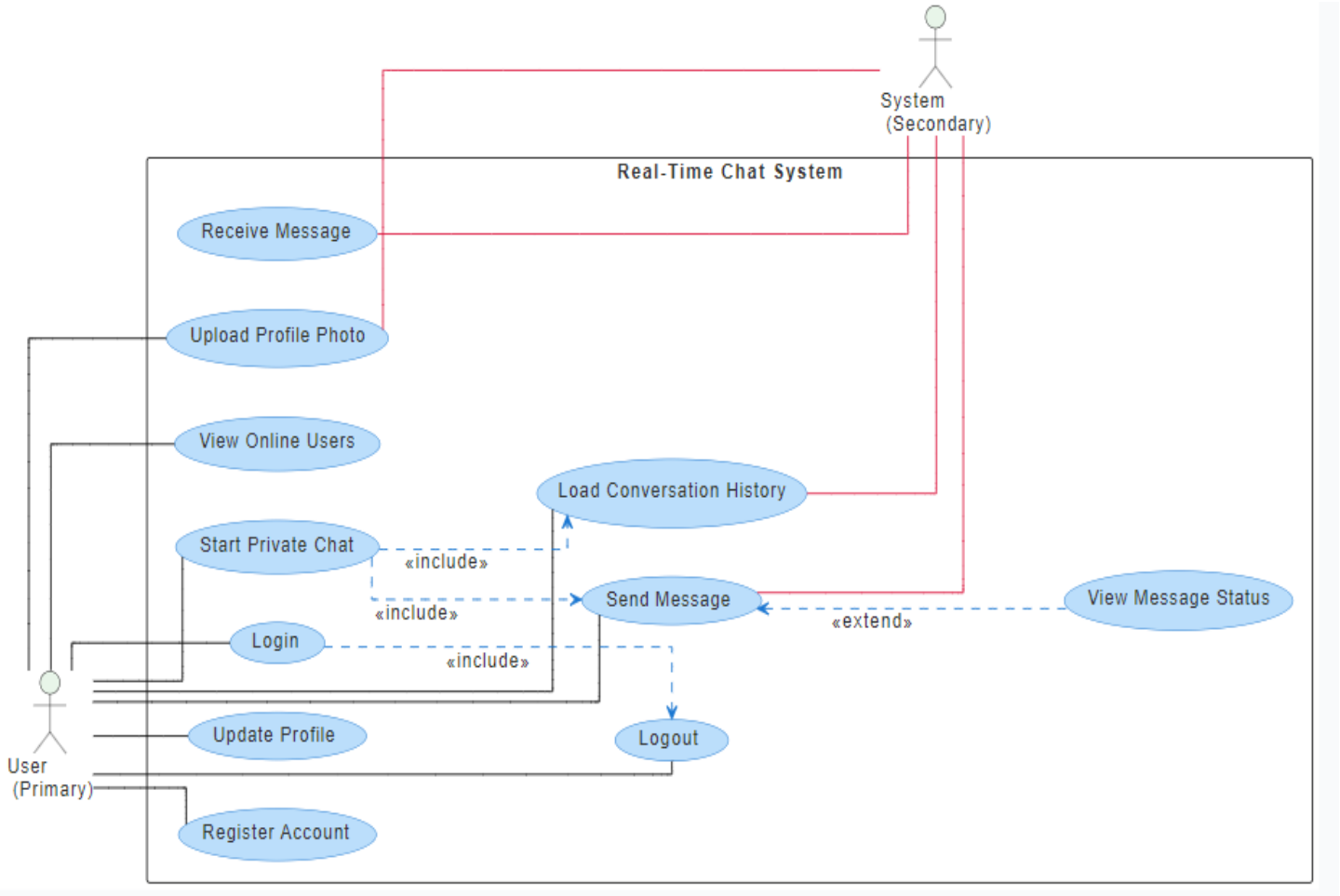
Use Cases

1. One-to-one real-time messaging
   Two users exchange instant private messages through WebSocket.
2. Reliable message delivery fallback
   If socket is unavailable, messages are still sent via API, so chat keeps working.
3. Conversation history retrieval
   Users open a contact and load older messages between both users.
4. Contact-based chat switching
   Users switch between multiple contacts and continue each separate conversation.
5. Profile management
   Users update display name and bio to personalize their identity.
6. Profile photo upload
   Users upload an avatar (JPG/PNG/WEBP/GIF) with file type and size validation.

Technical Specifications

- Frontend: React single-page app.
- Backend API: PHP REST-style endpoints.
- Realtime transport: WebSocket server using Ratchet.
- Database: MySQL
- Runtime target: Local Windows/Laragon development environment.

Use case Diagram



Task	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8
Requirement Analysis								
System Design (SRS, UML)								
Database Design (MySQL)								
Backend Development (PHP API)								
Frontend Development (React)								
WebSocket Integration								
Authentication Module								
User/Profile Module								
Chat/Messaging Module								
Testing & Debugging								
Deployment								
Documentation								

Technical Solution

- The site should be treated as a three-layer chat system: React frontend, PHP REST API, and a Ratchet-based WebSocket server. The frontend handles login, registration, user selection, chat UI, and profile editing. The API handles persistence for users, messages, and profile data. The WebSocket server handles real-time message delivery and presence updates.

Conclusion

This website is a solid real-time chat application prototype with the right core pieces in place: React for the interface, PHP for the API, MySQL for persistence, and WebSocket for live messaging. Its main value is in the real-time private chat flow, user profiles, and online/offline presence handling.